

Analysis of Algorithms Laboratory

Lab 10: Recursion, Dynamic Programming (Fibonacci), and 0–1 Knapsack

Aispur P. Santiago

School of Mathematical and Computational Sciences

Yachay Tech University

San Miguel de Urququí - Imbabura, Ecuador

April 27, 2026

1 Introduction

This laboratory explores the algorithmic and analytical differences between **plain recursion** and **dynamic programming**. The Fibonacci sequence is used as a first case study because it naturally exhibits *overlapping subproblems*: a naive recursive implementation recomputes the same values many times, leading to poor performance. Dynamic programming resolves this inefficiency by storing intermediate results (memoization) so that each subproblem is solved only once.

The second part of the laboratory addresses the **0–1 Knapsack Problem**, a classical optimization problem. Its structure is well-suited to dynamic programming because optimal solutions can be constructed from optimal solutions of smaller subproblems. The report focuses on formulation (state definition and recurrence), correctness reasoning, and complexity analysis.

2 Problem 1: Fibonacci by Plain Recursion (No Dynamic Programming)

2.1 Problem Statement

Compute the n th Fibonacci number:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2.$$

2.2 Algorithmic Idea

The recursive method directly translates the mathematical recurrence into code:

- If $n = 0$, return 0.
- If $n = 1$, return 1.
- Otherwise, return $F(n-1) + F(n-2)$.

2.3 Complexity Analysis

Time complexity. The recursion generates a tree of calls with extensive repetition. A standard bound is:

$$T(n) = T(n-1) + T(n-2) + \Theta(1) = \Theta(\varphi^n),$$

where $\varphi \approx 1.618$ is the golden ratio. Hence, the runtime is exponential.

Space complexity. The recursion depth is n , so the call stack requires:

$$S(n) = \Theta(n).$$

3 Problem 2: Fibonacci with Dynamic Programming (Memoization) — Full Solution

3.1 Problem Statement

Implement Fibonacci using dynamic programming (memoization). The objective is to ensure the program works correctly and avoids recomputation by storing intermediate results.

3.2 Methodology and Debugging Rationale

Memoization requires two essential conditions:

1. **A shared memory structure** (e.g., array `memo`) accessible to every recursive call.
2. **Correct recursive calls** that continue to pass the same memo structure (otherwise, cached values are not reused).

A common error in non-working memoized snippets is calling the recursion without passing `memo` (e.g., `fib(n-1)` instead of `fib(n-1, memo)`). This breaks memoization and restores exponential behavior (and may cause runtime errors depending on the language).

3.3 DP Formulation

Subproblem. Let $F(k)$ be the Fibonacci value for each $k \in \{0, 1, \dots, n\}$.

State storage. Maintain an array `memo` such that:

$$\text{memo}[k] = F(k) \text{ once computed.}$$

Entries not yet computed remain at a default value (commonly 0), with base cases handled explicitly to avoid ambiguity at $k = 0$.

Recurrence. For $n \geq 2$:

$$F(n) = F(n-1) + F(n-2).$$

With memoization, each $F(k)$ is computed once and then reused.

3.4 Correctness Sketch

We prove correctness by induction on n .

- **Base cases:** For $n = 0$ and $n = 1$, the algorithm returns the correct Fibonacci definitions.
- **Inductive step:** Assume the procedure returns correct values for all integers smaller than n . For $n \geq 2$, if `memo[n]` already stores a value, it equals $F(n)$ by construction. Otherwise, the algorithm computes $F(n-1)$ and $F(n-2)$ recursively (with memo reuse), which are correct by the inductive hypothesis. Their sum equals $F(n)$, which is stored and returned.

Thus, the memoized algorithm correctly returns $F(n)$.

3.5 Complexity Analysis

Time complexity. Each subproblem $F(k)$ for $k = 0, \dots, n$ is computed at most once, so:

$$T(n) = \Theta(n).$$

Space complexity. The memo table stores $n + 1$ values and the recursion depth is at most n :

$$S(n) = \Theta(n).$$

4 Problem 3: 0–1 Knapsack Using Dynamic Programming

4.1 Problem Statement

Given n items, item i has weight w_i and value v_i , and a knapsack capacity W . The objective is to maximize total value subject to the weight constraint:

$$\sum_{i \in S} w_i \leq W,$$

where each item can be chosen at most once (0–1 constraint).

4.2 DP State Definition

Define:

$$DP[i][c] = \text{maximum value achievable using the first } i \text{ items with capacity } c,$$

for $i = 0, \dots, n$ and $c = 0, \dots, W$.

4.3 Base Cases

$$DP[0][c] = 0 \ \forall c, \quad DP[i][0] = 0 \ \forall i.$$

4.4 Recurrence

For item i (1-indexed) with (w_i, v_i) :

$$DP[i][c] = \begin{cases} DP[i-1][c], & \text{if } w_i > c, \\ \max(DP[i-1][c], DP[i-1][c - w_i] + v_i), & \text{if } w_i \leq c. \end{cases}$$

4.5 Correctness Sketch

For each (i, c) there are exactly two possibilities:

- Exclude item i (value remains $DP[i-1][c]$),
- Include item i if feasible ($w_i \leq c$), yielding $DP[i-1][c - w_i] + v_i$.

Taking the maximum ensures optimality. Induction over i establishes correctness for the full table, and the final answer is $DP[n][W]$.

4.6 Complexity Analysis

The DP table has $(n + 1)(W + 1)$ entries and each entry is computed in $\Theta(1)$ time:

$$T(n, W) = \Theta(nW), \quad S(n, W) = \Theta(nW).$$

5 Complexity Summary

Table 1: Complexity Summary for Lab 10

Algorithm	Time Complexity	Space Complexity
Fibonacci (plain recursion)	$\Theta(\varphi^n)$	$\Theta(n)$
Fibonacci (memoized DP)	$\Theta(n)$	$\Theta(n)$
0–1 Knapsack (2D DP)	$\Theta(nW)$	$\Theta(nW)$

GitHub Repository

<https://github.com/icepursantiago/Analysis-of-Algorithms>

Appendix A: Provided Fibonacci Code (Unmodified)

The following program is included exactly as submitted; no changes were made.

Listing 1: Submitted C++ code: recursion vs memoized DP for Fibonacci (verbatim).

```
1  #include <iostream>
2  #include <chrono>
3  #define MAX 100
4  using namespace std;
5
6  int Fibonacci(int n) {
7      if (n == 0) { return 0; }
8      if (n == 1) { return 1; }
9      return Fibonacci(n - 2) + Fibonacci(n - 1);
10 }
11
12 int fib(int n, int memo[]) {
13     if (n == 0) { return 0; }
14     if (memo[n] != 0) {
15         return memo[n];
16     }
17     if (n == 1) {
18         memo[n] = 1;
19         return 1;
20     }
21     memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
22     return memo[n];
23 }
24
25 int print_fibonacci(int n) {
26     n++;
27     for (int i = 0; i < n; i++) {
28         cout << Fibonacci(i) << " ";
29     }
30     cout << endl;
31     return 0;
32 }
33
34 int main() {
35     int n;
36     int memo[MAX] = { 0 };
37
38     cout << "Enter the index number of the fibonacci number: ";
39     cin >> n;
40
41     if (n < 0 || n >= MAX) {
42         cerr << "n must be in range [0, " << (MAX - 1) << "]\n";
43         return 1;
44     }
45
46     // Time plain recursion
47     const auto t1 = chrono::high_resolution_clock::now();
48     const int r1 = Fibonacci(n);
49     const auto t2 = chrono::high_resolution_clock::now();
```

```

50
51 // Time memoized DP
52 const int r2 = fib(n, memo);
53 const auto t3 = chrono::high_resolution_clock::now();
54
55 const auto rec_us = chrono::duration_cast<chrono::microseconds>(t2
    - t1).count();
56 const auto dp_us = chrono::duration_cast<chrono::microseconds>(t3
    - t2).count();
57
58 cout << r1 << endl;
59 cout << "Dynamic Programming:" << endl;
60 cout << r2 << endl;
61
62 cout << "Time (plain recursion): " << rec_us << " us" << endl;
63 cout << "Time (memorized DP): " << dp_us << " us" << endl;
64
65 cout << "Fibonacci sequence:" << endl;
66 print_fibonacci(n);
67
68 return 0;
69 }

```